# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

| | | | |
|---|---|---|---|
| **Course Code:** | CSA1014 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO3 | **Assessment:** | Assessment Tool 1 – Git & Docker Technical Assignment |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO3 Statement:** | Build full-stack applications with an emphasis on containerization, version control, and collaborative development | | |
| **Student Name:** | Manish Kumar L | **Reg. No.:** | 1924260185 |
| **Date:** | 23/10/26 | **Duration:** | 60 minutes |

### Code of Conduct

***Manish Kumar L*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Manish Kumar L**

## Annexure A – Git & Docker Technical Assignment

**Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

    - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

### Rubrics for Calculation

| Evaluation Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Problem Analysis & Project Design(20%)** | Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification. | Good analysis and project structure with minor omissions. | Basic analysis and repository structure with limited justification. | Incomplete or unclear analysis and project organization. |

| | | | | |
|---|---|---|---|---|
| **Git Repository & Branching Strategy(20%)** | Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization. | Repository and branching strategy are mostly correct. | Basic Git setup with limited branching implementation. | Incorrect or incomplete Git repository and branching strategy. |
| **Version Control Workflow & Commit History(10%)** | Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history. | Most Git operations demonstrated correctly with good commit history. | Limited Git workflow and commit practices. | Poor workflow and inadequate commit history. |
| **Docker Implementation(20%)** | Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies. | Functional Docker implementation with minor issues. | Basic Docker implementation with limited explanation. | Incomplete or incorrect Docker implementation. |
| **Deployment, Testing & Results(15%)** | Application successfully deployed and tested with appropriate screenshots and evidence of successful execution. | Deployment completed with minor issues. | Partial deployment and testing demonstrated. | Deployment unsuccessful or insufficient evidence. |
| **Documentation, Challenges & Future Enhancements(15%)** | Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements. | Good documentation with minor omissions. | Basic documentation with limited discussion. | Poor documentation and insufficient analysis. |

| Criteria | **Mark** |
|---|---|
| System Requirement Analysis **(30%)** | /5 |
| Selection of Software Architecture **(25%)** | /5 |
| Architecture Diagram and Component Design **(20%)** | /5 |
| Component Interaction and Quality Attribute Analysis **(15%)** | /5 |
| Architectural Justification and Technical Presentation **(10%)** | /5 |
| **TOTAL** | /25 |

# Intelligent Software Requirements Traceability Platform

## Industry Context

**Large software projects contain thousands of requirements, design components, source-code modules, test cases, defects, and releases. Maintaining traceability manually is difficult.**

## Problem Statement

**Design and develop a platform that establishes traceability relationships between requirements, design artifacts, implementation components, test cases, defects, and releases.**

## Objectives

- **Manage software requirements.**
- **Link requirements with design.**
- **Link implementation with requirements.**
- **Map test cases to requirements.**
- **Track defects.**
- **Generate traceability matrices.**

## Expected Outcomes

- **Improved requirements visibility.**
- **Better test coverage.**
- **Reduced traceability gaps.**
- **Easier impact analysis.**
- **Improved software quality.**

# Git & Docker Technical Assignment

Large software development projects contain many requirements, design documents, source-code modules, test cases, defects, and software releases. Managing the relationships between these components manually is difficult and time-consuming. It can lead to missing requirements, incomplete testing, and difficulty identifying the impact of changes.

The Intelligent Software Requirements Traceability Platform is a web-based application that connects these software development components and generates traceability matrices. It helps developers, testers, and project managers monitor requirements from planning to release.

## Project Objectives:

- To create, update, view, and delete software requirements.
- To link requirements with design documents and implementation modules.
- To map test cases to their corresponding requirements.
- To record and track software defects.
- To generate requirement traceability matrices.
- To identify missing links and improve test coverage.
- To analyze the impact of requirement changes.

## Functional Requirements:

1. Requirement Management: Add, edit, view, and delete requirements.
2. Design Traceability: Connect each requirement with relevant design components.
3. Implementation Tracking: Link requirements with source-code modules.
4. Test Case Management: Associate test cases with requirements and record test results.
5. Defect Tracking: Record defects, severity, status, and related requirements.
6. Traceability Matrix: Generate a table showing relationships among requirements, design, implementation, and testing.
7. Impact Analysis: Identify related components when a requirement changes.

## Expected Outcomes

- Improved visibility of software requirements.
- Better test coverage and fewer missed requirements.
- Reduced traceability gaps.
- Easier change-impact analysis.
- Improved software quality and project maintenance.

## 2. Project Structure Design

The project repository is organized into separate folders for application code, templates, static files, tests, and Docker configuration.

Proposed Folder Structure

```
intelligent-traceability-platform/
│
├── app/
│   ├── __init__.py
│   ├── routes.py
│   ├── models.py
│   ├── templates/
│   │   ├── index.html
│   │   ├── requirements.html
│   │   ├── traceability.html
│   │   └── defects.html
│   └── static/
│       ├── css/
│       │   └── style.css
│       └── js/
│           └── script.js
│
├── tests/
│   └── test_app.py
│
├── requirements.txt
├── run.py
```

├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── .env.example
└── README.md

## Code:

```yaml
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://traceuser:${DB_PASSWORD:?Set DB_PASSWORD}@db:5432/tracedb
    depends_on:
      db:
        condition: service_healthy

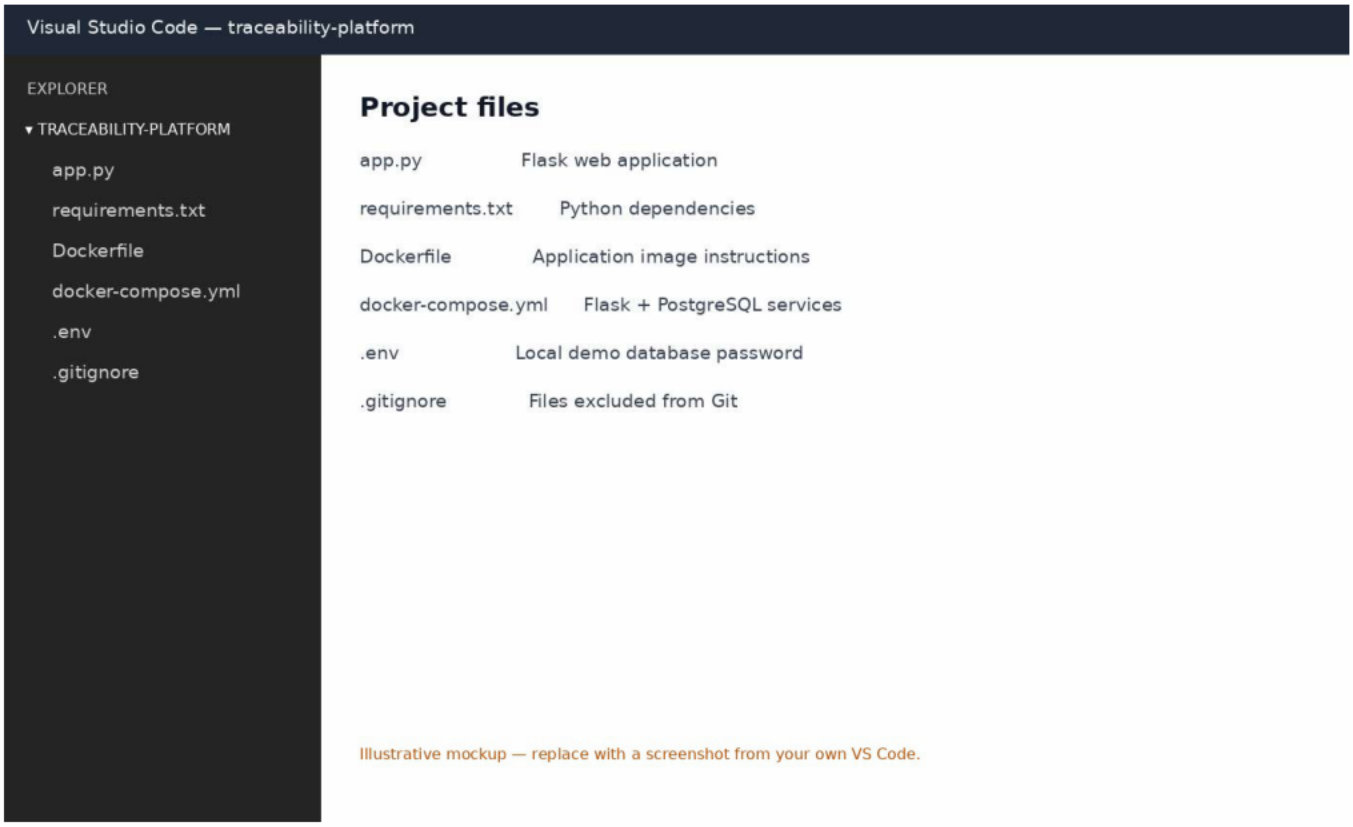
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: tracedb
      POSTGRES_USER: traceuser
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U traceuser -d tracedb"]
```


```yaml
    environment:
      POSTGRES_DB: tracedb
      POSTGRES_USER: traceuser
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U traceuser -d tracedb"]
      interval: 5s
      timeout: 3s
      retries: 10

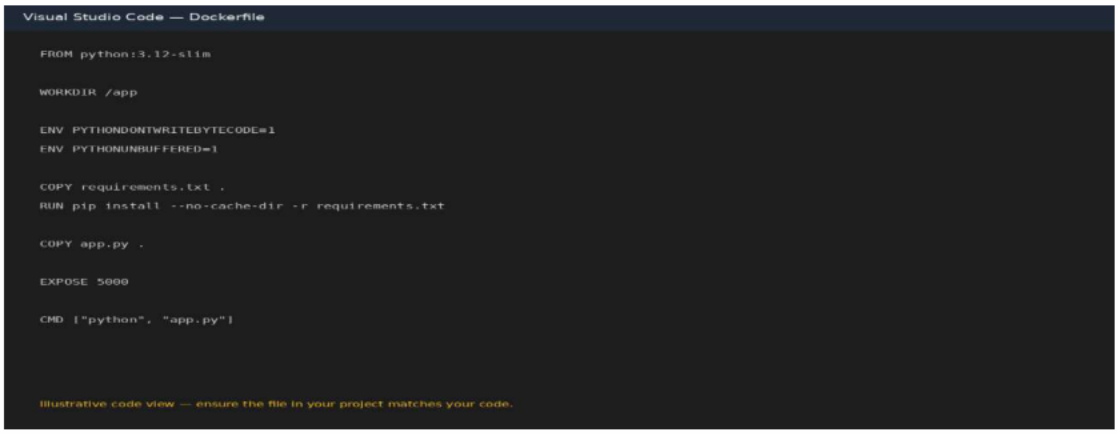
volumes:
  postgres_data:
```

# Output:

## 1-Project folder structure:




## 2- Dockerfile code:



# 3-Docker Compose configuration:

Visual Studio Code — docker-compose.yml

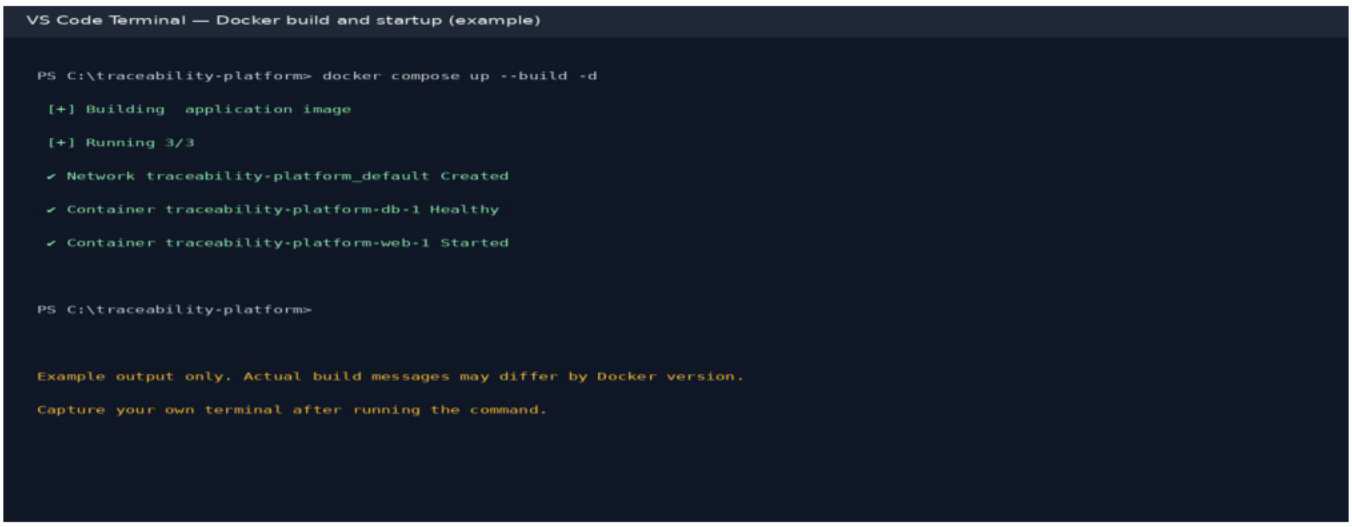
```
services:
  web:
    build: .
    ports: ["5000:5000"]
    environment:
      DATABASE_URL: postgresql://traceuser:${DB_PASSWORD:?Set DB_PASSWORD}@db:5432/tracedb
    depends_on:
      db:
        condition: service_healthy


  db:
    image: postgres:16
    environment:
      POSTGRES_DB: tracedb
      POSTGRES_USER: traceuser
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U traceuser -d tracedb"]
      interval: 5s
      timeout: 3s
      retries: 10
```

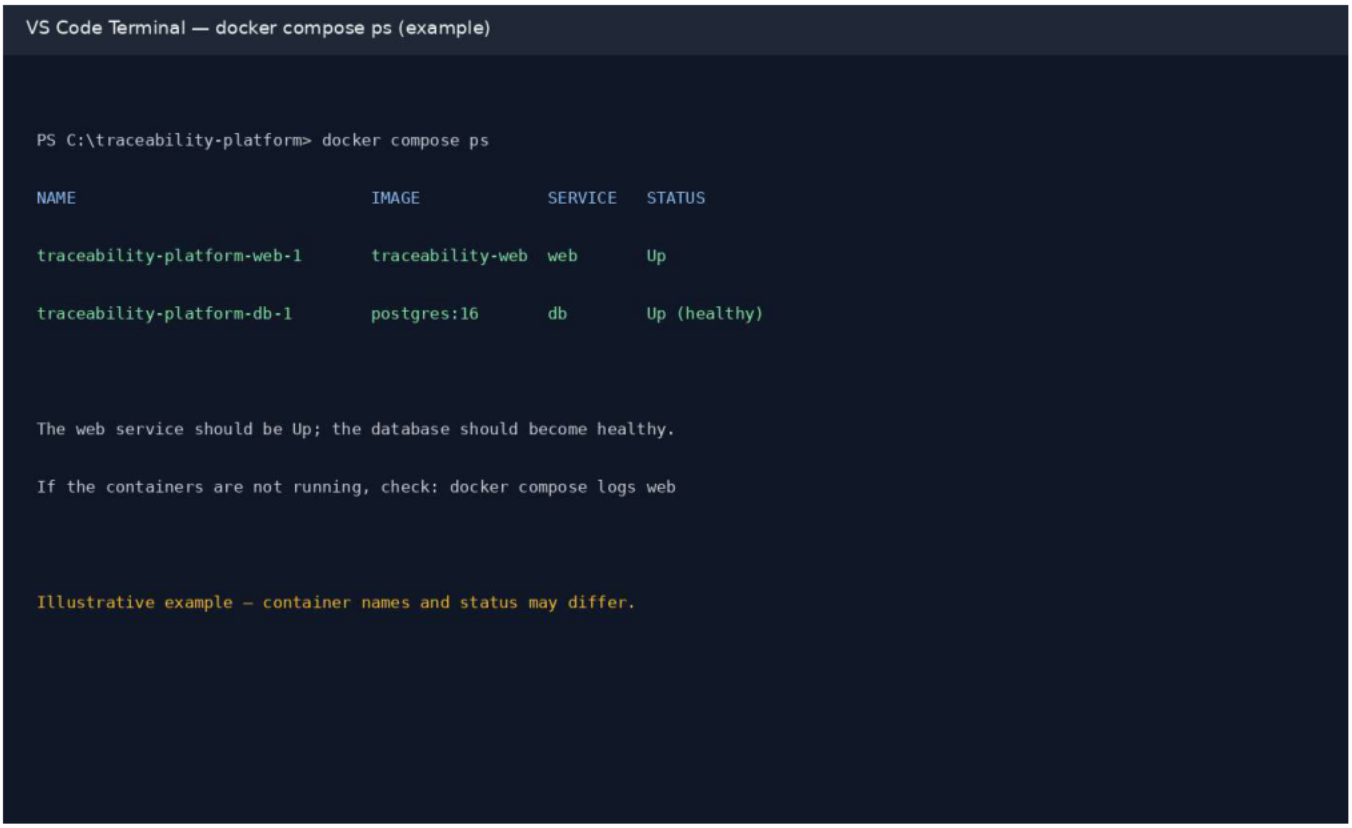
Illustrative code view — not a live VS Code capture.

```
volumes:
```

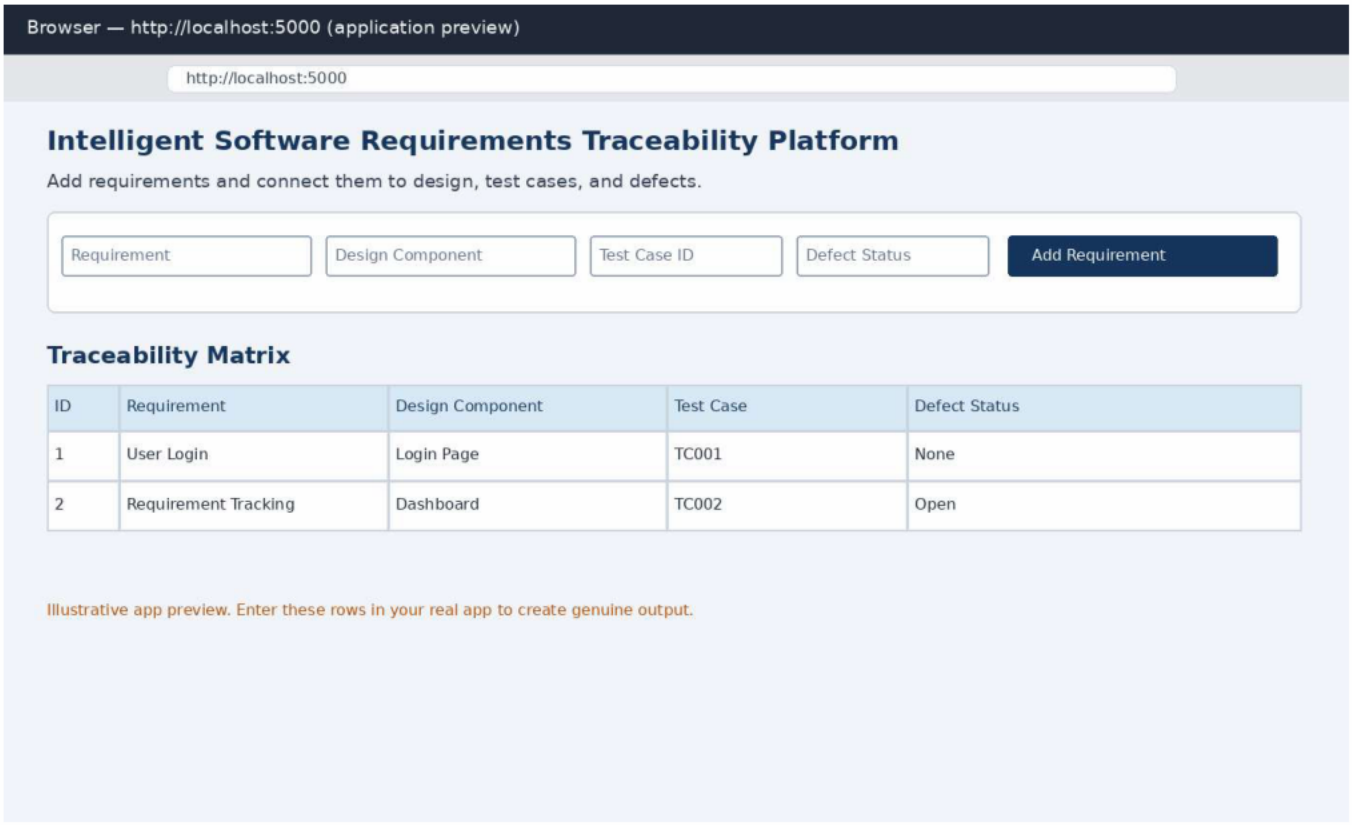
# 4-Docker build and startup:




# 5-Running Docker containers:



# 6-Web application output:




| Folder/File | Purpose |
|---|---|
| app/ | Contains the main application code. |
| routes.py | Defines pages and application operations. |
| models.py | Defines the data models for requirements, test cases, and defects. |
| templates/ | Contains the HTML pages displayed to users. |
| static/ | Stores CSS styles and JavaScript files. |
| tests/ | Contains automated application tests. |
| run.py | Starts the web application. |
| requirements.txt | Lists Python dependencies, such as Flask. |
| Dockerfile | Defines how to build the application image. |
| docker-compose.yml | Configures and starts the application and database services. |
| .gitignore | Prevents temporary files and secrets from being committed. |
| .env.example | Documents the environment variables needed to run the project. |

| | |
|---|---|
| README.md | Explains installation, execution, and project usage. |

This modular structure makes the application easier to understand, test, maintain, and extend.

## 3. Git Repository Initialization:

Git is a version control system used to record changes in the project's source code. GitHub can host the repository so that multiple developers can collaborate.

Steps to Initialize the Repository

Open Command Prompt or a terminal and execute the following commands.

```
mkdir intelligent-traceability-platform
cd intelligent-traceability-platform

git init
git branch -M main

git add .
git commit -m "Initial project structure"
```

- README.md — documents the project and setup instructions.
- requirements.txt — records the Python packages needed to run the application.
- Git commits — provide snapshots of project changes.
- Git remote — connects the local repository to GitHub.

Result: Git is initialized, the project files are committed, and the repository can be synchronized with GitHub.

## 4. Git Branching Strategy:

The project uses a feature-based branching strategy with main, develop, feature branches, release branches, and hotfix branches.

main

Stable, tested application versions

develop

Integration of completed features

Feature branches

Requirements, tests, defects

Release branches

Final testing and fixes

Hotfix branch

Urgent fixes for production issues

Branch Descriptions

- Main branch: Contains stable versions of the application.
- Development branch: Combines completed features before release.
- Feature branches: Used to develop individual features, such as requirement management and traceability matrices.
- Release branches: Used for final testing, bug fixes, and release preparation.
- Hotfix branches: Used to correct urgent issues in the stable version.

## 5. Version Control Workflow:

The Git workflow records the development of the application through meaningful commits and branch merges.

Workflow Steps

1. Create the repository and commit the initial project structure.
2. Create a feature branch for requirement management.
3. Implement the requirement management functionality.

4. Stage and commit the changes.
5. Merge the feature branch into develop.
6. Repeat the process for test cases, defects, and traceability matrices.
7. Merge the tested release into main.
8. Push the changes to GitHub.

A merge conflict may occur when two branches modify the same lines of a file.

Steps to resolve it:

1. Run git status to identify conflicting files.
2. Open the conflicting file.
3. Review the changes and combine the correct code.
4. Remove the conflict markers.
5. Stage and commit the resolved file.

```
git add app/routes.py
git commit -m "Resolve merge conflict in application routes"
```

Conflicts should only be demonstrated if one actually occurs. This workflow helps prevent accidental overwriting and maintains consistent project code.

## 6. Commit History Analysis:

Commit history provides a record of how the project has developed. Each commit should describe one logical change.

Example Commit History

```
a1b2c3d  Initial project structure
b2c3d4e  Add requirement management module
c3d4e5f  Implement design and code traceability
d4e5f6a  Add test case and defect tracking
e5f6a7b  Generate requirement traceability matrix
f6a7b8c  Add Docker configuration
g7b8c9d  Configure Docker Compose services
h8c9d0e  Add application tests and documentation
```

## 7. Docker Environment Design:

Docker is a containerization platform that packages an application with its runtime and dependencies. It allows the same application to run consistently on different computers.

Why Docker Is Suitable

- It reduces differences between development and deployment environments.
- It simplifies application setup.
- It packages the Python runtime and required libraries.
- It makes the application easier to share with team members.
- It supports repeatable deployment and future scaling.

Components to Containerize

| **Component** | **Purpose** |
|---|---|
| Flask web application | Provides the user interface and backend operations. |
| Database | Stores requirements, design links, test cases, and defects. |
| Test environment | Runs automated tests before deployment. |

For a practical student implementation, Flask and PostgreSQL can run as separate containers. The database stores persistent project information, while Flask handles application requests.

## 8. Dockerfile Development:

The Dockerfile defines the steps required to build a container image for the Flask application.

Dockerfile

Create a file named Dockerfile in the project root.

```
FROM python:3.12-slim

WORKDIR /app
```

```
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "run.py"]
```

# 9. Docker Compose Design:

Docker Compose is used to define and run multiple containers using a single YAML configuration file.

docker-compose.yml

The following configuration uses Flask and PostgreSQL.

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://traceuser:${DB_PASSWORD:?Set DB_PASSWORD}@db:5432/tracedb
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: tracedb
      POSTGRES_USER: traceuser
```

```
    POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U traceuser -d tracedb"]
    interval: 5s
    timeout: 3s
    retries: 10
  restart: unless-stopped

volumes:
  postgres_data:
```

Create a local .env file in the project root containing a strong development password:

```
DB_PASSWORD=replace_with_a_strong_local_password
```

Do not commit .env to Git. Add it to .gitignore, and do not use a development password in a public deployment.

## 10. Container Deployment and Testing:

The project is deployed locally using Docker Compose.

### Step 1: Build and Start Containers

Open a terminal in the project root and set the database password using your terminal's environment-variable syntax, or provide it through the ignored .env file.

Run:

```
docker compose config
docker compose up --build -d
```

The first command validates the Compose configuration. The second builds the Flask image and starts the services in the background.

## Step 2: Verify Running Containers

docker compose ps

Both the web and db services should be running, and the database should report a healthy status.

## Step 3: Check Application Logs

docker compose logs web
docker compose logs db

These commands help identify errors in the Flask application or PostgreSQL startup.

## Step 4: Open the Application

Open the following address in your browser:

Local application URL

**http://localhost:5000**

Available after the web service starts successfully.

Open

## Step 5: Test Application Functions

| Test | Expected Result |
|---|---|
| Open the home page | Dashboard loads successfully. |
| Add a requirement | Requirement is saved. |
| Link a design component | The relationship appears in the application. |
| Add a test case | Test case is associated with a requirement. |
| Record a defect | Defect information is saved. |
| Generate a traceability matrix | Relevant links and missing relationships are displayed. |
| Restart containers | Application runs and stored database data remains available. |

These are expected results, not completed test evidence. Execute the tests and capture the actual outputs for your submission.

### Step 6: Stop the Application

`docker compose down`

This stops and removes the containers while preserving the named database volume. Avoid `docker compose down -v` unless you intentionally want to delete the stored database volume.

Screenshots Required for the Report

Capture screenshots of the following:

1. Git repository and project folder structure.
2. Feature branch creation and merging.
3. Actual Git commit history.
4. Docker image build output.
5. `docker compose ps` showing the running services.
6. Application dashboard open in the browser.
7. Requirement creation and traceability matrix.
8. Database persistence after restarting the containers.

These screenshots provide evidence that the implementation and deployment were actually performed.

## 11. Benefits of Git and Docker

| Area | Benefits for This Project |
|---|---|
| Collaboration | Git allows developers to work on separate modules using feature branches. |
| Version management | Commit history records changes to requirements, tests, and application code. |
| Portability | Docker packages the application and its dependencies. |
| Deployment | Docker Compose starts the web application and database together. |
| Scalability | Services can be scaled or deployed on suitable infrastructure as demand grows. |
| Maintainability | Modular source code, meaningful commits, and containerized environments simplify updates. |
| Testing | Automated tests and repeatable container environments improve reliability. |

| | |
|---|---|
| Recovery | Git enables code rollback, while database backups help recover stored project information. |

## 12. Challenges and Improvements

## Challenges Encountered or Anticipated

- Git merge conflicts: Different developers may modify the same application files.
- Dependency issues: Different Python or library versions may cause errors.
- Database connectivity: The application may start before PostgreSQL is ready.
- Environment configuration: Missing or incorrect database passwords may prevent startup.
- Data persistence: Removing database volumes can cause data loss.
- Traceability gaps: Some requirements may not be linked to test cases or implementation components.
- Testing difficulties: It may be difficult to confirm that all relationships are correct.

These are common implementation challenges. Include only the ones you actually encountered in the final report.

## Possible Improvements:

1. Add automated GitHub Actions workflows to run tests on every pull request.
2. Introduce code reviews before merging changes into `main`.
3. Add role-based access for developers, testers, and project managers.
4. Implement automatic detection of requirements without test cases.
5. Add graphical traceability matrices and change-impact visualizations.
6. Introduce database backup and recovery procedures.
7. Use separate development, testing, and production configurations.
8. Add application logging and monitoring.
9. Use secure secret management for deployment.
10. Deploy the containerized application to a cloud platform when required.

These enhancements can make the system more reliable, secure, and suitable for larger software projects.

## Conclusion

The Intelligent Software Requirements Traceability Platform provides a centralized way to manage software requirements and their relationships with design components, implementation modules, test cases, defects, and releases.

Git supports collaboration, branching, change tracking, and version recovery. Docker and Docker Compose provide a consistent environment for running the web application and database. By combining requirement traceability, version control, and containerization, the project can improve testing visibility, reduce missing relationships, simplify change-impact analysis, and support better software quality.